MINI PROJECT 10

Secure Customer KYC Verification API

Comprehensive Requirements Specification

Goal: assess application design, secure coding, API design, data modelling, testing, and engineering discipline.

Assignment type	Recommended duration	Team size	Expected level
Full-stack secure web/API project	3-5 weeks	1-3 students	Intermediate

Technology freedom

Students may use Java/Spring Boot with Angular/React, or an equivalent modern stack. A relational database is recommended. The submitted solution must run locally using documented steps and must not depend on paid services.

1. Project Overview

Business objective: Design an API-first KYC document workflow that securely accepts evidence, permits tightly controlled retrieval, records verification outcomes, and minimizes exposure of personally identifiable information.

Primary actors: Customer, KYC Analyst, Compliance Supervisor, API Client Administrator

Minimum viable scope:

- Upload KYC documents
- List and retrieve authorized KYC documents
- Submit verification result
- Retrieve customer verification status
- Audit all access and decisions

Learning outcomes

- Translate a business brief into a layered and testable solution.
- Design normalized data models and clear RESTful contracts.
- Apply authentication, authorization, validation, audit logging, and secure error handling.
- Demonstrate critical business rules with automated tests.
- Communicate architecture decisions, assumptions, risks, and limitations professionally.

2. Scope and Boundaries

The following capabilities are mandatory. Features not listed may be implemented as extensions after the core acceptance criteria pass.

Capability	Minimum requirement
F1	Upload KYC documents
F2	List and retrieve authorized KYC documents
F3	Submit verification result
F4	Retrieve customer verification status
F5	Audit all access and decisions

Out of scope

- Production deployment, commercial payment, identity, notification, antivirus, or e-signature services.
- Real personal, financial, health, academic, or identity information. Use synthetic test data only.
- Native mobile application. Responsive web UI or API client demonstration is sufficient.
- Advanced analytics, machine learning, and large-scale distributed architecture unless selected as an extension.

3. Users, Roles, and Authorization

Role	Required access
Customer	Own records and permitted self-service operations.

Role	Required access
KYC Analyst	Assigned workflow and review operations within authorized scope.
Compliance Supervisor	Assigned workflow and review operations within authorized scope.
API Client Administrator	User/role configuration, reference data, and support operations; no unrestricted business-data access by default.

Authorization must be enforced in the backend for every protected operation. Hiding a button in the UI is not an authorization control. Test horizontal privilege escalation and vertical privilege escalation explicitly.

4. Functional Requirements and Workflows

ID	Workflow	Detailed requirement
FR-01	Case creation	Authorized client creates or retrieves a KYC case for a customer using an idempotency key.
FR-02	Document upload	Client uploads an allowed document type; service validates metadata and content, encrypts/stores it, and returns an opaque document ID.
FR-03	Analyst verification	Authorized analyst records checks and a decision with reason codes; supervisor approval is required for configured high-risk outcomes.
FR-04	Status retrieval	Authorized client obtains the current case state and sanitized decisions, not raw internal notes or unrelated documents.

Core business rules

- KYC documents and identifiers are highly sensitive; use least privilege and minimize returned fields.
- Document retrieval requires case-level authorization and must not expose direct storage URLs.
- Verification decision changes must be versioned, attributable, and immutable in audit history.
- Support idempotency for case creation and document metadata submission to prevent duplicates.
- Logs must mask customer identifiers and must not contain document bytes, access tokens, or full identity numbers.

Common UI expectations

- Login page and role-appropriate landing page.
- List pages with loading, empty, error, filtering, and pagination states.
- Create/update forms with client-side convenience validation plus authoritative server-side validation.
- Details page showing current status and allowed next actions.
- Accessible confirmation and error messages without revealing technical internals.

5. Data Model Requirements

Entity	Minimum data considerations
Customer	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.
KycCase	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.

Entity	Minimum data considerations
KycDocument	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.
DocumentMetadata	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.
VerificationCheck	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.
VerificationDecision	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.
ApiClient	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.
AuditEvent	Stable primary key; created/updated timestamps; required domain fields; constraints and relationships appropriate to the use case.

Data modelling rules

- Use foreign keys and database constraints for important invariants.
- Use optimistic locking/versioning or equivalent protection where concurrent updates matter.
- Define deletion behavior. Prefer status or archival for auditable business records.
- Store timestamps consistently and state the timezone policy.
- Provide schema migration scripts and seed only fictional development data.

6. API and Integration Contract

Method and path	Purpose / expectation
POST /api/v1/kyc/cases	Implement authorization, request validation, consistent response structure, and documented error cases.
POST /api/v1/kyc/cases/{caseId}/documents	Implement authorization, request validation, consistent response structure, and documented error cases.
GET /api/v1/kyc/cases/{caseId}/documents	Implement authorization, request validation, consistent response structure, and documented error cases.
GET /api/v1/kyc/cases/{caseId}/documents/{documentId}	Implement authorization, request validation, consistent response structure, and documented error cases.
POST /api/v1/kyc/cases/{caseId}/verification-results	Implement authorization, request validation, consistent response structure, and documented error cases.
GET /api/v1/kyc/cases/{caseId}/status	Implement authorization, request validation, consistent response structure, and documented error cases.

API standards

- Use JSON for structured requests and responses; use multipart only for file upload.
- Use appropriate HTTP methods and status codes. Distinguish validation, authentication, authorization, not-found, conflict, and server errors.
- Return a correlation ID in errors. Do not return stack traces or SQL messages.
- Use pagination metadata for collections and allow-list sortable fields.

- Publish an OpenAPI specification with example requests, responses, and security scheme.

7. Non-Functional Requirements

Quality attribute	Requirement
Security	Use salted adaptive password hashing, role-based authorization, secure session or token handling, server-side validation, output encoding, CSRF protection where cookie sessions are used, and safe error messages.
Performance	For normal list operations, target p95 response time below 2 seconds with at least 25 concurrent test users; use pagination for all potentially large collections.
Reliability	Use database transactions for multi-step business changes. Define consistent rollback behavior and prevent duplicate processing using uniqueness or idempotency where relevant.
Auditability	Record authentication events, access denials, important data changes, document access, workflow decisions, actor, timestamp, and correlation ID. Never log passwords, tokens, or sensitive file content.
Usability	Provide clear validation messages, accessible labels, keyboard-friendly forms, confirmation for destructive actions, and responsive layouts.
Maintainability	Use layered architecture, clear naming, centralized exception handling, configuration outside code, automated migrations, and meaningful unit/integration tests.
Privacy	Collect only data necessary for the stated features. Restrict sensitive fields in API responses and mask them in screens and logs.

8. Security Requirements

Security is part of the functional definition, not an optional enhancement. The design should follow defense in depth and least privilege.

- Password policy: minimum 12 characters for locally managed accounts; store only adaptive salted hashes. Seeded demo accounts must require documented non-sensitive passwords.
- Session/token policy: expiration, logout invalidation where applicable, secure cookie flags for browser sessions, and no tokens in URLs or logs.
- Input validation: define length, format, range, enum, file, date, and cross-field rules on the server.
- Authorization: verify role, ownership, relationship, and current workflow state for each request.
- Data protection: mask sensitive values, restrict response fields, avoid sensitive query strings, and document encryption assumptions.
- Error handling: generic user-facing messages, detailed server diagnostics without secrets, and consistent error schema.
- Dependency hygiene: include a dependency inventory and resolve known high-severity issues where feasible.
- Abuse protection: throttle login and other sensitive operations; handle repeated and automated requests safely.

Required negative security tests

- Authentication bypass and weak password/session handling
- Broken object-level authorization by changing resource identifiers
- Broken role authorization by calling privileged endpoints directly
- SQL/NoSQL injection through form, query, sort, and filter parameters
- Stored and reflected cross-site scripting where text is displayed

- CSRF for state-changing browser requests when cookies are used
- Unsafe file upload, path traversal, content-type mismatch, and oversized files
- Sensitive data exposure in logs, URL parameters, errors, browser cache, and API responses
- Race conditions or duplicate submissions for critical operations
- Missing rate limits on login and high-risk operations

9. Acceptance Criteria

ID	Acceptance criterion	Evidence expected
AC-01	Unauthorized API client receives no case existence information beyond a safe error response.	Automated test and/or reproducible demonstration with screenshots or API evidence.
AC-02	Unsupported, oversized, or content-mismatched files are rejected.	Automated test and/or reproducible demonstration with screenshots or API evidence.
AC-03	Repeated idempotent request does not create duplicate cases or metadata records.	Automated test and/or reproducible demonstration with screenshots or API evidence.
AC-04	Raw documents are not returned in list responses and never use public URLs.	Automated test and/or reproducible demonstration with screenshots or API evidence.
AC-05	Decision history identifies actor, role, timestamp, reason codes, and prior state.	Automated test and/or reproducible demonstration with screenshots or API evidence.

Definition of done

- All mandatory features and acceptance criteria pass.
- No known critical or high-severity authorization, injection, XSS, file handling, or data-exposure issue remains.
- Application starts from a clean checkout using the documented procedure.
- Database migrations, sample configuration, tests, and API documentation are included.
- Repository contains no secrets, real personal information, generated build folders, or unnecessary binaries.

10. Testing Expectations

Test level	Minimum expectation
Unit tests	Business rules, validators, state transitions, calculations, and error branches.
Repository/integration tests	Database constraints, queries, transactions, and concurrency-sensitive behavior.
API tests	Happy paths, invalid inputs, authentication, authorization, ownership, pagination, and conflicts.
UI tests	At least two critical end-to-end journeys, if a UI is built.
Security tests	Negative tests listed in Section 8, including direct API calls that bypass UI controls.
Quality target	Meaningful coverage of critical logic; coverage percentage alone is not accepted as evidence of quality.

11. Architecture and Implementation Guidance

Recommended logical architecture

Browser or API Client -> Web/API Layer -> Application/Service Layer -> Domain and Validation -> Repository -> Relational Database

Cross-cutting components: authentication, authorization, audit logging, exception handling, observability, configuration, and file storage where required.

- Keep controllers thin. Business rules belong in service/domain components.
- Use DTOs at the API boundary rather than exposing persistence entities directly.
- Centralize mapping, validation, authorization policies, and exception-to-response handling.
- Use database transactions around atomic business operations.
- Use generated opaque IDs or perform strict authorization if sequential IDs are used.
- For file-based projects, keep metadata in the database and content in protected storage outside the public web root.

Suggested technology baseline

Area	Example choices
Backend	Java 21+, Spring Boot, Spring Security, Bean Validation, JPA/Hibernate
Frontend	Angular or React; alternatively, an API-only submission with a complete Postman/Bruno collection
Database	PostgreSQL, MySQL, or H2 only for tests
Documentation	OpenAPI/Swagger plus README and architecture note
Testing	JUnit, Mockito, Spring Boot Test, Testcontainers where available, and frontend test framework
Packaging	Maven or Gradle; optional Docker Compose for local database

12. Deliverables

- Source repository with readable commit history.
- README: prerequisites, setup, run, test, demo accounts, assumptions, and troubleshooting.
- Architecture document: context, components, data flow, trust boundaries, and two or more key decisions.
- ER diagram and database migrations.
- OpenAPI specification and API test collection.
- Automated test suite and short test report.
- Security note: threat model, controls implemented, known limitations, and dependency scan summary.
- Five-to-ten-minute demonstration script or recorded demo, if requested by the instructor.

13. Evaluation Rubric

Category	Weight	What good looks like
Functional correctness	25%	Complete core workflows, correct rules, robust edge cases, and consistent state.
Security and privacy	25%	Backend authorization, safe input/output, secure files/sessions, no secret or sensitive-data leakage.

Category	Weight	What good looks like
Architecture and code quality	20%	Layered design, maintainable code, clear domain model, transactions, and sound error handling.
Testing	15%	Focused unit, integration, API, and negative security tests for critical behavior.
API/UI usability	10%	Clear contracts, validation messages, pagination, accessibility, and coherent user journeys.
Documentation and delivery	5%	Reproducible setup, diagrams, assumptions, decisions, and professional presentation.

Critical-failure rule: A solution with a reproducible authentication bypass, cross-user data access, unsafe document exposure, committed secret, or destructive data integrity flaw cannot receive a high overall rating until corrected.

14. Optional Extensions

- Email or in-app notification abstraction with a local fake implementation.
- Docker Compose packaging and environment-specific configuration.
- Optimistic locking and conflict-resolution UX.
- Export selected records to CSV or PDF with authorization checks.
- Search indexing or caching with documented consistency behavior.
- CI pipeline for build, tests, linting, dependency scan, and artifact generation.
- Basic observability dashboard using structured logs and application metrics.

Extensions are assessed only after the mandatory requirements are functional, secure, tested, and documented.

15. Student Submission Checklist

- [ ] Application builds and starts from a clean checkout.
- [ ] All mandatory workflows are demonstrated.
- [ ] Roles and test users are documented.
- [ ] Authorization is enforced by the backend.
- [ ] Known invalid and malicious inputs are tested.
- [ ] No credentials, tokens, or real personal data are committed.
- [ ] Schema migrations and fictional seed data are included.
- [ ] OpenAPI and API collection are current.
- [ ] Automated tests pass.
- [ ] Architecture, threat model, assumptions, and limitations are documented.

Instructor note: Students should be encouraged to explain trade-offs rather than adding unnecessary technologies. A smaller solution with correct security boundaries, data integrity, and tests is preferable to a broad but fragile solution.